

VeriFlow

AI 出题与 workflow 可核验、可追溯

规格驱动验证 · 运行证据 · 受约束修复

面向出题质检与流程审核，将模型生成的节点图
转换为可检查的约束、反例证据与发布门禁。

第八届 AIC 算法创新赛 · AI+ 软件创新

评审阅读版 / 算法详解 / 20 页 / 2026 年 10 月



从 workflow 生成到修复、发布，再到程序判题

模型输出可编辑 workflow；验证器给出问题、见证、修复决策与发布门禁。

01 规格驱动的出题质检

自然语言要求与平台规则编译为 Spec；IR 描述实际节点与连接。
结构、语义、数据流、可达与安全五维检查输出统一证据。

02 运行证据与受约束修复

模拟轨迹逐条验证时序义务，补足静态检查观察不到的故障。
规则或 AI 提出补丁；六阶段守卫筛选，拒绝无改善与回归。

03 解题与工程交付

C++17 / Python3 沙箱、隐藏数据、对拍与失败反例。
提供 Web 演示、部署说明、核心算法代码与测试材料。

主要算法：图路径搜索、时序监视、轨迹对齐、候选筛选与增量复验

competition-v2 / 55 组内部案例

1.000	检测 F1
35/50	定位正确
38/45	修复后静态 PASS

5 组正常 + 50 组注入故障
5 条运行专有故障不进入修复分母

检查 AI 工作流的缺步骤、错连接与运行中断

生成的流程可能缺步骤、连错数据或执行中断；一次模型自评难以给出稳定证据。

实际风险	VeriFlow 检查机制	评委可复核产物
缺失审核或必需动作	动作数量检查 + 绕过路径 BFS	缺失动作、节点与见证路径
数据连接与类型不一致	生产者可达性 + 边级类型检查	期望 / 实际类型、关联边
图完整，执行轨迹中断	时序义务监视 + 轨迹对齐	事件位置、缺失动作、轨迹片段
自动修复引入新风险	补丁守卫 + 全量复验 + 回归拒绝	补丁、候选拒绝原因、图差异

评分对应 创新性 20：验证与修复闭环 技术 20：算法实现 功能 20：双角色产品
方案 20：需求至部署证据 应用 15：出题质检与 CI 展望 5：边界与后续验证

两条流程覆盖出题质检与程序判题

出题者检查 workflow 并决定是否发布；解题者提交代码，查看判题结果与反例。

出题者

需求与题目规格	生成 / 编辑 IR	静态 + 运行验证	受约束修复	门禁与入库
自然语言要求	节点、边、表达式	问题列表与轨迹	补丁与差异	READY / 拦截
动作与时序约束	模型提案可修改	审核门 / 范围守卫	拒绝则保留原图	审核后发布

解题者

选题与样例	提交代码	Docker 沙箱	隐藏数据 / 对拍	反例与反馈
读取题面与数据	C++17 / Python3	编译、限时执行	期望与实际比较	输入、输出、来源
样例通过不等于正确	统一任务接口	隔离运行与资源约束	保留失败分类	按证据重新提交

共同产物：验证报告、执行记录、问题证据与可复核决策；流程图存在不等于实际执行完成。

模型生成 JSON，后端完成验证、模拟与修复

前端编辑与展示流程；后端接收 IR 和 Spec，返回验证报告、运行记录及修复结果。

Next.js Web

需求编辑、节点图、证据工作台、修复与基准页面

FastAPI / 结构化任务接口

IR / Spec → 检查、模拟、修复 → 报告 / GateResult

Python 验证与修复内核

ir / spec / staticcheck / verify / runtime / explain / repair
共享数据模型；保留版本、约束 ID、节点与见证路径

执行与存储

Docker 沙箱：提交判题 SQLite：任务与验证记录

AI 的使用位置

DeepSeek / OpenAI 兼容接口
用于 workflow 生成、可选修复提案
与解题教练；调用记录可追溯。

选择与判定边界

兼容接口便于替换模型与配置。
Spec 默认编译器使用关键词 / 模板。
模型输出先校验结构，再运行规则。
模型评分不作为发布门禁。
无密钥时可运行确定性案例与修复。

Spec 规定要求, IR 描述实际节点与连接

同一需求可对应不同流程图; 核验的是图是否满足显式约束, 而非图像是否相似。

WorkflowSpec / 应当发生什么

```
"required_actions": [{
  "id": "review", "kind": "human_gate",
  "cardinality": "at_least_one" }],
"ordering_constraints": [{
  "id": "o1", "before": "human_gate",
  "after": "publish_problem" }],
"safety_policies": [{ "id": "p1",
  "kind": "require_human_gate" }]
```

另含 `data_dependencies` 与 `temporal_constraints`。
`source_traces` 记录约束对应的需求或平台政策。
默认 `compiler=heuristic`, 采用关键词 / 领域模板。

数据流: 需求 → `compile_spec` → `check_spec`; 生成 JSON → WorkflowIR 校验 → `verify_workflow`。

WorkflowIR / 实际如何连接

```
"nodes": [
  {"id": "gen", "kind": "tool",
   "tool": "test_generator"},
  {"id": "pub", "kind": "tool",
   "tool": "publish_problem"}
],
"edges": [{"from": "gen", "to": "pub"}]
// 缩略字段; 此连接绕过了人工审核
```

节点还可含 `expr`、`on_fail`、输入 / 输出类型与配置。
Pydantic 拒绝多余字段、重复 ID 和未知节点引用。
Spec 查互逆顺序、未知选择器引用和数量冲突。

五类检查返回统一的问题列表和约束结果

collect_issues 调度检查, assemble_result 汇总约束判定、严重度与关联证据。

维度	检查事实	代表性错误 / 输出
结构	节点关系、环路、守卫表达式与失败分支	环路见证 / MISSING_ON_FAIL
语义	必需动作数量与所有发布路径的前置关系	MISSING_REQUIRED_ACTION / ORDER_VIOLATION
数据流	生产者到消费者可达、相邻边类型匹配	BROKEN_BINDING / TYPE_MISMATCH
可达	节点能否沿有向路径到达终点	DEAD_NODE + 关联节点
安全	审核门、范围、凭据与外发策略	MISSING_HUMAN_GATE / HARDCODED_SECRET

状态汇总优先级

failed_constraint or HIGH/CRITICAL → FAIL
else issues → WARNING; else unknown → UNKNOWN; else PASS

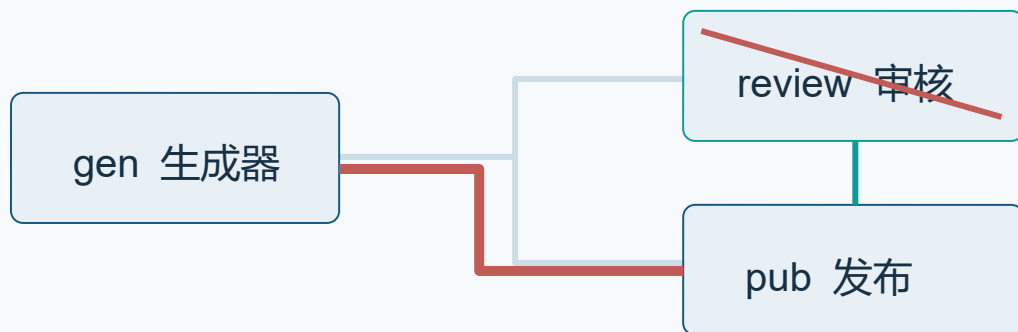
统一 Issue 对象

code / severity / constraint_id / affected_nodes
expected / actual / witness_path / evidence

屏蔽审核节点后，BFS 寻找绕过路径

若去掉审核节点后仍能从源节点到达发布节点，就得到一条违反前置要求的路径。

反例图 / 存在审核门仍可能绕过



屏蔽 review 后仍有 $gen \rightarrow pub$ ：判定失败

1. 收集 human_gate，作为 blocked 集合。
2. 从原图入度为 0 的非屏蔽节点出发。
3. BFS 跳过 blocked；parent 保存前驱。
4. 命中目标后回溯，输出 witness_path。

bypass_path / 核心伪代码

```
Q ← sources(G) - blocked
while Q:
    u ← pop_left(Q)
    if u == target: return backtrack(parent,u)
    for v in outgoing[u]:
        if v not in blocked and v not in seen:
            parent[v] ← u; seen.add(v); Q.push(v)
    return None
```

cycle_witness / 三色 DFS

WHITE 未访；GRAY 在栈；BLACK 已完成。
遇到指向 GRAY 的边，用 parent 回溯闭合环。
BFS 与 DFS：时间 $O(V+E)$ ，辅助空间 $O(V)$ 。

不同规则检查数量、路径、类型与策略

必需动作、节点关系、数据类型和配置策略分别检查；报告说明期望值和实际发现。

语义：选择器匹配 + 数量 + 顺序

`match_nodes` 可按节点 `id`、`kind` 或 `tool` 匹配。

`at_least_one` 要求数量 ≥ 1 ；`exactly_one` 要求 $=1$ 。

顺序要求检查是否有绕过前置节点的目标路径；

无绕过时，再确认前置节点确实能够到达目标。

```
count(match("human_gate")) == 0  
→ MISSING_REQUIRED_ACTION / expected  $\geq 1$ 
```

数据流：依赖存在与相邻类型

生产者与消费者之间至少存在一条有向路径。

边两端均声明类型时，比较 `out_type` / `in_type`。

例：`object` $\rightarrow$ `string`，输出 `TYPE_MISMATCH`。

结果保留 `expected` / `actual` / `affected_nodes` / `constraint_id`，供定位、解释与修复共用。

安全：配置扫描 + 领域策略

工具名按领域白名单检查，非法工具被拒绝。

凭据键识别 `api_key` / `token` / `password` 等。

非空字符串且非环境引用 $\rightarrow$ `HARDCODED_SECRET`。

外发 URL 与范围守卫按政策 / 关键词检查。

```
config.api_key = "literal"  $\rightarrow$  CRITICAL  
config.api_key = "env:API_KEY"  $\rightarrow$  引用形式
```

当前检查边界

类型检查比较顶层 `type`，未证明完整 `schema` 兼容。

路径存在不能证明字段值实际正确传递。

安全扫描不等同污点分析或形式化安全证明。

运行监视检查动作是否发生、顺序与次数是否满足

monitor_trace 读取完成事件；动作存在于图中，并不能代替轨迹中的执行证据。

ExecutionTrace / 事件样例

```
{ "event_index": 2, "node_id": "if_pay",  
  "node_type": "branch",  
  "operation": "branch",  
  "status": "success", "branch": "true" }  
// source="mock": 确定性模拟 DAG 遍历
```

仅 success / mocked 事件参与动作命中。

按 node_id、node_type 或 operation 选择事件。

重复完成事件分别计数，保留重试与重复写入。

BEFORE、次数、最终发生与条件义务分别判定。

未观测到要求的分支 → UNKNOWN。

IF_BRANCH_THEN / 条件义务



```
last_b ← max(consequence_hits) or None  
unmet ← [i for i in trigger_hits  
         if last_b is None or i ≥ last_b]  
unmet 非空 → FAIL；报告最早未满足的触发位置
```

一次较晚的结果可满足多次先前触发。

失败输出事件位置、轨迹片段与反例。

门禁拦截运行 FAIL 或约束可判定覆盖不足。

实现范围：10 类时序算子子集；模拟外部写入，不等于真实 n8n 执行或完整 LTL 模型检查。

先构造线性扩展，再对齐预期与观测序列

无前后约束的并发节点按观测顺序优先排列，降低将合法交换误记为顺序错误的机会。

align_trace / 实现链路

- ① 对每个节点搜索下游可达集合。
- ② 用规格顺序约束补充下游关系。
- ③ 按 IR 边拓扑排序，就绪项按观测位置排序。
- ④ DP 最小化删除 / 插入，回溯形成对齐行。
- ⑤ 对匹配行做局部逆序检查，附加分支偏差。

示例：运行中缺少审核动作

expected: gen → review → pub

observed: gen → pub

review → MISSING_EXPECTED; edit_cost = 1

编辑距离递推 / Needleman–Wunsch 式

```
D(i,0)=i; D(0,j)=j
D(i,j)=min(D(i-1,j)+1, D(i,j-1)+1,
D(i-1,j-1)+cost)
cost=0 if expected[i]==observed[j] else 106
同 ID 匹配; 不同 ID 优先拆为缺失+额外执行
```

按当前实现分阶段核算

可达集合: $O(V \cdot (V+E))$, 空间最多 $O(V^2)$ 。

就绪列表反复排序: 保守 $O(V^2 \log V+E)$ 。

DP: 时间与空间 $O(nm)$, n/m 为两序列长度。

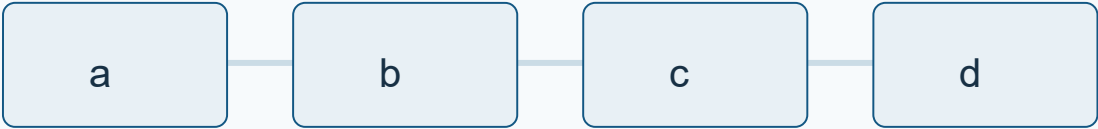
另有规格关系补充; 以上不是整体性能实测。

实现边界: 采用单个线性扩展与局部检查; 未做完整并发最优对齐, 也不保证检出全部逆序。

保留违反约束的路径，再裁剪解释所需节点

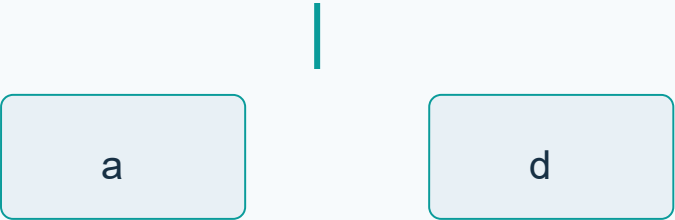
报告把“违反哪条约束、涉及哪些节点、看到了什么”连在一起，方便人工复核与补丁定位。

证据切片示意 / 非可执行子流程



完整 witness_path : $a \rightarrow b \rightarrow c \rightarrow d$

affected_nodes : a、d ; 聚焦端点



minimized_nodes 聚焦 a、d ; 原见证仍单独保存。

minimized_edges 保留端点间原有边及关联边。

裁剪后可能不连通，不把它当作可运行的新图。

minimize_issue / 核心步骤

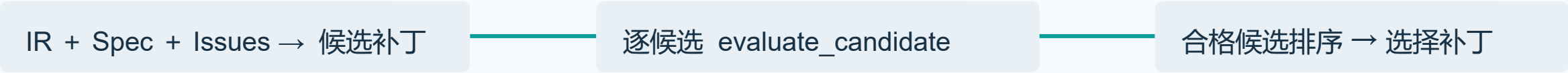
```
keep ← unique(witness_path + affected_nodes)
keep ← keep n known_node_ids
core ← endpoints + affected on witness
若 core 至少 2 节点: 优先保留 core
edges ← retained_edges + affected_edges
```

输出与限制

issue_id / constraint_ids / original_node_count
minimized_nodes / edges / witness_path
method = witness-slice+endpoint-reduction
globally_minimal = false
未枚举子图并重验，未证明全局最小反例。

修复候选必须通过守卫与全量复验

默认每轮最多选择 3 个候选：规则计划变体与可选 AI 提案共享同一套筛选流程。



阶段	具体判定	失败处理
1 补丁结构	结构化 Patch 对象，拒绝空补丁列表	拒绝、后续跳过
2 图完整性	节点 / 边引用、操作前后置条件合法	拒绝、后续跳过
3 策略白名单	新增或修改的工具必须属于当前领域白名单	拒绝非法工具
4 增量筛选	目标已解决，或问题 / 失败约束数量减少	无改善则拒绝
5 全量验证	对修改后的图完整运行 verify_workflow	交给回归判定
6 回归决策	目标消失，无新增高风险；可达 / 结构类别未从无变有	接受或拒绝

接受补丁可仅解决目标问题，最终仍可能 FAIL；循环继续修复，发布时另由门禁独立判定。

按风险与改动量排序，按验证器类别重算

先满足修复条件，再选择改动较小的候选；增量筛选之后仍执行全量验证。

lex_key / 从左到右比较，越小越优

(目标未修复?, 新增高风险数,
失败约束数, 新增可达问题类别?,
改动节点数, 改动边数,
改动参数数, 补丁操作数)
先筛合格候选, 再按此元组排序选第一项。

verify_repair_loop / 停止条件

初始 PASS：无需修复；每轮重新收集问题。
接受候选后替换当前图，记录 patch 与 diff。
达到 PASS、候选被拒绝、故障指纹重复即停止。
另设迭代上限：默认 3 轮，竞赛基准使用 2 轮。

workflow_changes → impact_set → rerun

识别变更 ——— 传播至下游 ——— 选择验证器

参数配置变更 → safety。
类型绑定变更 → dataflow + semantic。
条件变更 → structural + semantic + safety。
节点 / 工具类型 / 边变化 → 全五维。

无历史结果或需重算全五维 → full fallback。
否则重算选中类别，复用其他类别的问题结果。
影响集合辅助说明，未逐约束精确缩减执行范围。

“改动较小”由候选内词典序选择实现；不宣称全局最小补丁，通用加速倍数尚未测量。

直接入库路径被拦截，两轮补丁后放行

需求要求生成器、范围守卫、审题门与入库；原图只有 $\text{gen} \rightarrow \text{pub}$ 两个节点。

原图与失败证据



```
MISSING_HUMAN_GATE
witness_path = ["gen", "pub"]
MISSING_REQUIRED_ACTION: guard
WEAK_BOUNDS: 缺少上界守卫
```

静态 FAIL / 发布 BLOCKED

审核集合为空；BFS 不需枚举全部路径。
直接得到 $\text{gen} \rightarrow \text{pub}$ ，作为拦截依据。

规则修复实测 / allow_ai=False

第 1 轮：插入 review，替换直连边

```
add_node(review); disconnect(gen, pub)
connect(gen, review); connect(review, pub)
```

第 2 轮：插入 g_bounds，补上上界约束

```
expr: spec.n_min ≥ 1 && spec.n_max ≤ 100000
gen → g_bounds → review → pub
```

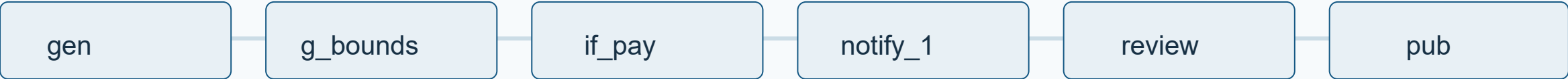
最终：静态 PASS / 门禁 READY

每轮 4 个操作，共 8 个；最终增加 2 个节点。
第一轮接受补丁后仍有缺陷，继续修复。
两轮后完整验证与模拟运行均满足门禁。

分支后断流触发运行门禁，静态仍通过

6 个节点、5 条边均在图上；模拟执行设置 skip_after="if_pay"，仅记录前 3 个事件。

完整 IR 的节点顺序



实际轨迹

gen@0 → g_bounds@1 → if_pay@2(true)

后续未执行：通知、审核与入库

运行义务 / 5 项 FAIL

EVENTUALLY publish_problem：未出现在轨迹。
EXACTLY_ONCE publish_problem：执行次数 0。
IF_EXECUTED_THEN：生成后缺少审核。
IF_BRANCH_THEN：true 后缺少通知。
EVENTUALLY notify：通知未出现在轨迹。

静态与运行交叉结果

静态：12/12 约束通过，五个维度均 PASS。
轨迹对齐：缺失 3 个预期节点，对齐代价 3。
节点覆盖：3/6；时序可判定覆盖：7/7。
覆盖可判定不代表约束通过，5 项已判定失败。

Gate：BLOCKED / reason=RUNTIME_FAIL

运行报告与沙箱判题展示可复核证据

左为 case4 断流报告；右为 VF1001 真实 WA 记录，可核对输入与输出。

① 运行轨迹区 / 仅记录 3 个事件

运行轨迹

已记录 · 3 个事件

gen 成功 · success

g_bounds 成功 · success

if_pay 成功 · success

当前问题相关轨迹停在 if_pay · true · 3 个相关事件

点击事件可查看关联节点；本例轨迹在 if_pay 的 true 分支后结束。

② 问题证据区 / 发布动作没有执行

证据

runtime.temporal

预期

eventually publish_problem

实际

not in trace

反例路径

gen → g_bounds → if_pay

解题沙箱 / 真实 WA 记录

C++17 / Python3 ， 容器隔离执行
VF1001 ： 输入与错误输出可复核

输入测试数据 (stdin)

3
1 2 3

期望标准输出 (expected)

6

实际程序输出 (actual)

666

50 条故障检出， 35 条定位正确， 38 条修复后通过

5 组正常流程 + 50 组注入故障；检测、诊断、定位与修复分别计分，不混为同一准确率。

指标	结果 / 计数	判定含义与样本范围
检测 P / R / F1	1.000 / 1.000 / 1.000	50 条故障检出； 5 条正常无误报； FN=FP=0
诊断正确率	1.000 / 50/50	问题码匹配套件约定的目标故障
定位正确率	0.700 / 35/50	命中套件标注的被改动位置； 15 条未正确定位
最终静态通过率	0.844 / 38/45	两轮后 38 条静态 PASS ； 5 条运行故障不计入分母

结果解释

$F1=2PR/(P+R)$ ； P 与 R 均为 1，故 $F1=1$ 。
检出异常仍可能未命中修改位置，定位需继续改进。
合成套件结果不能外推为未知流程的全部准确率。

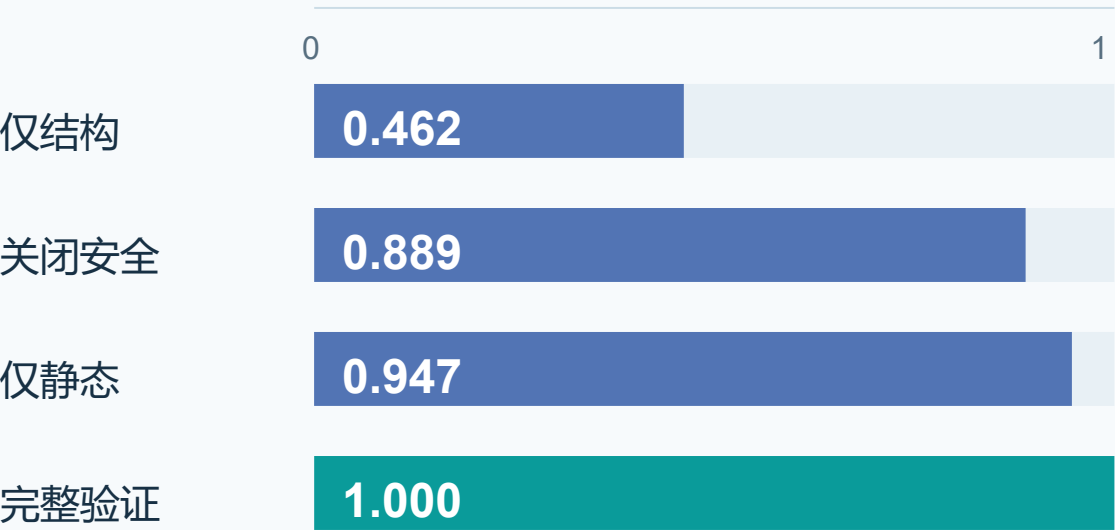
拒绝补丁与性能口径

missing_branch / broken_binding： 6 条无改善。
另 1 条 broken_binding 补丁不合法，保留拒绝。
静态均值 0.175 ms； 监视均值 0.114 ms（进程内）。

关闭验证层会增加漏检，测试与部署记录可复核

四种配置使用同一套 55 组案例，精确率均为 1； F1 差异来自故障漏检。

检测 F1 / 取值 0-1



只做结构：召回 0.300，漏掉语义等故障。
关闭安全：召回 0.800，运行监视仍开启。
仅静态：召回 0.900，漏掉 5 条运行专有故障。

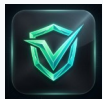
工程记录与复现入口

历史 pytest：287 passed，2026-09-30 记录。
前端构建与 Playwright 未在该报告中重跑。
公网：Ubuntu 24.04，Nginx 8081 统一入口。
Next.js 3000 / FastAPI 8010 / Docker。

`python scripts/competition_benchmark.py`

未运行项与增量样本

大模型对照：未运行，评测密钥未设置。
竞赛套件的增量加速：未测量。
另有 4 节点历史单例，非竞赛增量测量。
单例结果一致、未回退；缺重复次数与分布。
仅作初步记录，不能据此宣称通用加速倍数。



可用于出题质检与 CI 门禁，扩展需继续验证

目前已实现需求编译、 workflow 验证、 轨迹监视、 补丁修复和发布门禁。

已实现 / 可使用场景

出题质检：识别缺审核、弱范围与连接错误，保留失败证据。

流程门禁：结构化报告与退出码可接入 CI；失败时阻断发布。

解题训练：沙箱、隐藏数据与反例帮助复现错误。

当前边界 / 已明确限制

内部合成基准，无真实用户收益或规模化生产数据支持。

安全检查与并发对齐均有近似；运行验证采用模拟执行。

真实 n8n / Dify、Java、交互题、特殊评判与 C++ AST 变异未接入。

未来规划 / 验证优先

扩充真实缺陷与陌生流程样本，重点提高定位与补丁覆盖。

重复测量增量耗时、等价性与回退率，完成大模型对照。

接入真实轨迹，验证并发语义；开展用户试点与产品体验评估。

演示入口： <http://116.62.5.67:8081/>

复核材料： `docs/contest / packages / examples/golden / experiments/runs/competition`